

Manual de Resolução do Trabalho 01: Mundo dos Blocos de Tamanho Variável via SAT Solver

Fundamentos de Inteligência Artificial — Prof. Edjard Mota

Alunos: André Cavalcanti, Tiago Gil de Brito, Ygor Gutierrez, Sofia Braga

2 de outubro de 2026

Sumário

1	Introdução ao Problema	1
2	Descrição Formal do Mundo dos Blocos de Tamanho Variado	1
2.1	Domínio	1
2.2	Representação em LPO	1
2.3	Elementos novos e ações associadas (adds/deletes)	2
2.4	Ordem parcial ($A \xrightarrow{P} B$)	2
2.5	Resolução manual	2
3	Codificação CNF	4
3.1	Variáveis proposicionais	4
3.2	Grupos de cláusulas	4
4	Exemplos dos 3 cenários codificados para CNF	5
4.1	Situação 1, meta S_{f4} , $T = 4$	5
4.2	Situações 2 e 3	5
5	Mapeamento: Descrição Formal $\rightarrow$ Código	6
6	Execução Passo a Passo	6
7	Interpretação da Saída do SAT Solver	6

1 Introdução ao Problema

O Mundo dos Blocos clássico tem blocos de tamanho 1 e uma única relação de apoio. Aqui os blocos têm *comprimentos diferentes* (a, b : 1uc; c : 2uc; d : 3uc), ocupam posições horizontais livres sobre uma mesa de 6 slots, podem deixar espaços vazios, deslizar lateralmente e ficar em *ponte* sobre dois apoios. Um bloco só é estável se tiver apoio suficiente. O objetivo é, dado um estado inicial S_0 e um estado-meta, obter um plano (sequência de ações *move*) e provar sua otimalidade com um SAT solver.

O trabalho cobre três cenários (Situações 1, 2 e 3 do enunciado). O fluxo seguido foi: (i) modelagem em lógica de primeira ordem (LPO); (ii) resolução *manual* dos cenários; (iii) codificação proposicional (CNF); (iv) execução do SAT solver e interpretação; (v) comparação com os planos manuais e com um planejador BFS independente (oráculo).

2 Descrição Formal do Mundo dos Blocos de Tamanho Variado

2.1 Domínio

$B = \{a, b, c, d\}$; $L = B \cup \{\top\}$ (locais de apoio, $\top$ = mesa); $\ell(a) = \ell(b) = 1$, $\ell(c) = 2$, $\ell(d) = 3$. O eixo tem os pontos $X = \{0, \dots, 6\}$ e os slots $s_i = [i, i + 1]$, $i \in \{0, \dots, 5\}$ (6 slots, não 7). Um bloco que começa no ponto p cobre os slots $\{p, \dots, p + \ell(b) - 1\}$; as posições válidas são $p \in \{0, \dots, 6 - \ell(b)\}$. Os níveis são 0 (mesa) a 3.

2.2 Representação em LPO

Usamos o cálculo de situações: s é uma situação e $do(\alpha, s)$ a situação após a ação α .

Fluentes primitivos. $at(b, p, s)$: b começa no ponto p ; $lev(b, l, s)$: b está no nível l .

Relações derivadas (definidas por equivalência, nunca armazenadas):

$$\begin{aligned} cov(b, k, l, s) &\leftrightarrow \exists p. at(b, p, s) \wedge lev(b, l, s) \wedge p \leq k < p + \ell(b) \\ free(k, l, s) &\leftrightarrow \neg \exists x. cov(x, k, l, s) \\ on(b, y, s) &\leftrightarrow \exists k. cov(b, k, l, s) \wedge cov(y, k, l - 1, s) \quad (\text{e } on(b, \top, s) \leftrightarrow lev(b, 0, s)) \\ clr(b, s) &\leftrightarrow \neg \exists x, k. cov(b, k, l, s) \wedge lev(b, l, s) \wedge cov(x, k, l + 1, s) \\ stable(b, p, l, s) &\leftrightarrow l = 0 \vee \# \{k \in [p, p + \ell(b) - 1] : \exists y \neq b. cov(y, k, l - 1, s)\} \geq \lceil \ell(b)/2 \rceil \end{aligned}$$

Como on é derivada, um bloco pode ter vários apoios (ponte): $on(d, a, s) \wedge on(d, b, s)$.

Ação. $move(b, y, p)$: “mover b para cima de $y \in (B \setminus \{b\}) \cup \{\top\}$, começando no ponto p ”. Seja $l_y(s) = 0$ se $y = \top$ e $l_y(s) = l' + 1$ se $lev(y, l', s)$ (nível de destino).

$$\begin{aligned} Poss(move(b, y, p), s) &\leftrightarrow clr(b, s) \wedge \neg (at(b, p, s) \wedge lev(b, l_y(s), s)) \\ &\quad \wedge (y \in B \rightarrow \exists k \in [p, p + \ell(b) - 1]. cov(y, k, l_y(s) - 1, s)) \\ &\quad \wedge \forall k \in [p, p + \ell(b) - 1]. \forall x \neq b. \neg cov(x, k, l_y(s), s) \\ &\quad \wedge stable(b, p, l_y(s), s) \wedge l_y(s) \leq 3 \end{aligned}$$

Axiomas de estado sucessor, para $\alpha = move(b, y, p)$ e $l = l_y(s)$:

$$\begin{aligned} at(x, q, do(\alpha, s)) &\leftrightarrow (x = b \wedge q = p) \vee (at(x, q, s) \wedge x \neq b) \\ lev(x, m, do(\alpha, s)) &\leftrightarrow (x = b \wedge m = l) \vee (lev(x, m, s) \wedge x \neq b) \end{aligned}$$

2.3 Elementos novos e ações associadas (adds/deletes)

Elemento	Tipo	Add	Delete
$\ell(b)$, B , X	termos/constant—	—	— (rígidos)
$at(b, p)$	fluente	$at(b, p)$	$at(b, p_0)$ antigo
$lev(b, l)$	fluente	$lev(b, l_y + 1)$ (ou 0)	$lev(b, l_0)$ antigo
$clr(y)$	derivado	—	deixa de valer se b passa a cobrir y
$clr(z)$, z antigo	derivado	vale se nada mais o cobre	—
apoio			
$on(b, y)$	derivado	novos apoios sob $[p, p + \ell(b))$	apoios antigos
$stable$	pré-condição	—	—
$free(k, l)$	derivado	células deixadas por b	células ocupadas por b

Só *at* e *lev* são alterados pela ação; tudo o mais é recalculado, o que elimina os axiomas de frame para *clr* e *on*. *Correção em relação ao esboço do manual*: não se exige *clr(y)* para o destino *y*; basta que os slots-alvo estejam livres. Exigir *clr(y)* tornaria inalcançáveis os estados com dois blocos sobre *c* (por exemplo S_7).

2.4 Ordem parcial ($A \xrightarrow{P} B$)

Um plano é uma ordem parcial de ações com *ligações causais* $A \xrightarrow{P} B$: *A* produz *P* e *B* consome *P*, sem que nenhuma ação intermediária desfaça *P*. Em nossa representação, *P* é uma *celula* (slot, nível) ou a posição de um bloco. Para cada ação $\alpha = \text{move}(b, y, p)$ definimos:

$$W(\alpha) = \{\text{pos}(b)\} \cup \text{celulas_antigas}(b) \cup \text{celulas_novas}(b)$$

$$R(\alpha) = \{\text{pos}(y)\} \cup \text{celulas_acima_antigas}(b) \cup \text{apoio_novo}(b) \cup \text{celulas_novas}(b)$$

$\alpha_i \prec \alpha_j$ se $i < j$ e $W_i \cap R_j \neq \emptyset$ (ligação causal $\alpha_i \xrightarrow{P} \alpha_j$), $W_i \cap W_j \neq \emptyset$ ou $R_i \cap W_j \neq \emptyset$ (proteção). Ações sem relação são *independentes*: qualquer ordem é válida. O fecho transitivo é reduzido (diagrama de Hasse) em `interpretar.py`, que também *verifica* que toda linearização é executável.

2.5 Resolução manual

Notação: $x@p/l$ = bloco *x* no ponto *p*, nível *l*. Em todo passo foram verificados: topo livre, slots-alvo livres no nível de destino e apoio $\geq \lceil \ell/2 \rceil$. Os cenários estão desenhados abaixo.

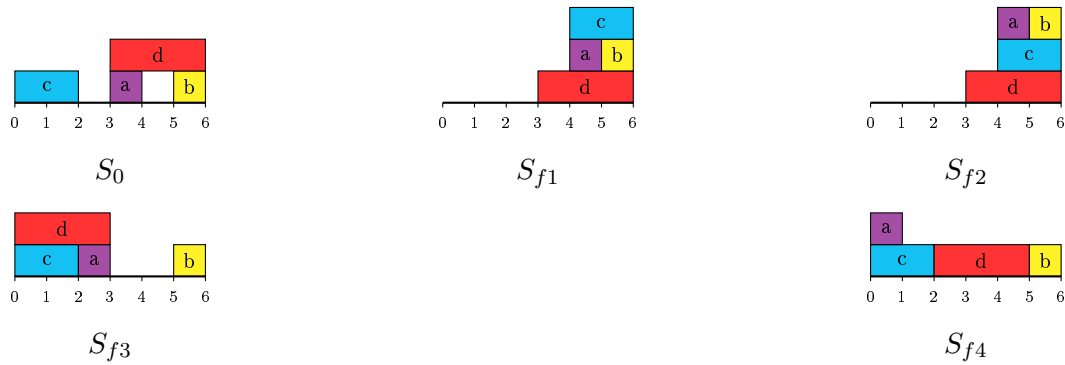


Figura 1: Situação 1: S_0 e S_{f1} – S_{f4} .

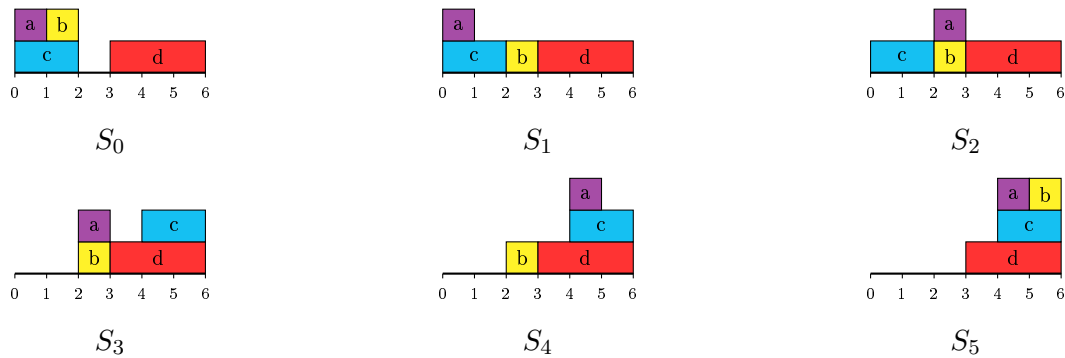


Figura 2: Situação 2: S_0 a S_5 .

Situação 1 (S_0 : $c@0/0$, $a@3/0$, $b@5/0$, $d@3/1$ — *d* em ponte sobre *a* e *b*).

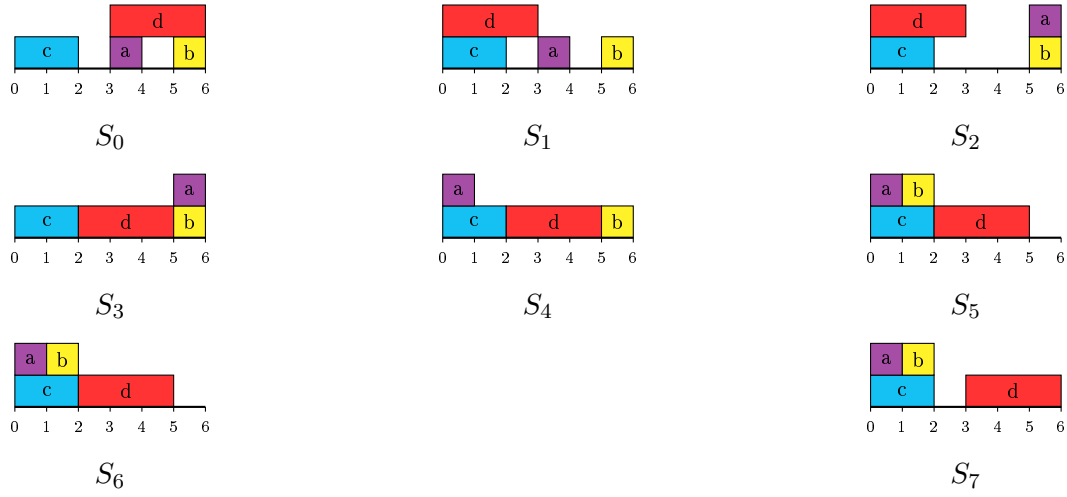


Figura 3: Situação 3: S_0 a S_7 (S_5 e S_6 coincidem na figura do enunciado).

Meta	Plano manual	Ações
S_{f3}	$move(d, c, 0); move(a, T, 2)$	2
S_{f4}	$move(d, c, 0); move(a, b, 5); move(d, T, 2); move(a, c, 0)$	4
S_{f1}	$move(d, c, 0); move(a, T, 2); move(d, a, 1); move(b, c, 0); move(d, T, 3); move(a, d, 4); move(b, d, 5); move(c, a, 4)$	8
S_{f2}	$move(c, T, 1); move(d, c, 0); move(a, T, 0); move(d, c, 1); move(b, a, 0); move(d, T, 3); move(c, d, 4); move(b, c, 5); move(a, c, 4)$	9

Execução detalhada de S_{f4} :

t	Ação	Verificação e novo estado
0	$move(d, c, 0)$	$clr(d)$; slots 0–2 do nível 1 livres; apoio: slots 0,1 (c) = 2 $\geq$ 2. $d@0/1$
1	$move(a, b, 5)$	$clr(a)$; slot 5 do nível 1 livre; apoio b . $a@5/1$
2	$move(d, T, 2)$	slots 2–4 do nível 0 livres (a saiu do slot 3). $d@2/0$
3	$move(a, c, 0)$	slot 0 do nível 1 livre; apoio c . $a@0/1 \Rightarrow S_{f4}$

Observação: o plano de exemplo do esboço do manual (“ a para a mesa em $p = 4$ e depois d para a mesa em $p = 2$ ”) é inválido, pois d cobriria o slot 4, ocupado por a .

Situação 2 (S_0 : $c@0/0$, $d@3/0$, $a@0/1$, $b@1/1$), meta S_5 : $d@3/0$, $c@4/1$, $a@4/2$, $b@5/2$.

t	Ação	Novo estado / observação
0	$move(a, d, 3)$	$a@3/1$ (apoio: slot 3 de d)
1	$move(b, a, 3)$	$b@3/2$ (estaciona b sobre a)
2	$move(c, d, 4)$	c ficou livre; $c@4/1$, slots 4,5 sobre d
3	$move(b, c, 5)$	$b@5/2$
4	$move(a, c, 4)$	$a@4/2 \Rightarrow S_5$

Situação 3 (S_0 igual ao da Situação 1), meta S_7 : $c@0/0$, $a@0/1$, $b@1/1$, $d@3/0$.

t	Ação	Estado do enunciado atingido
0	$move(d, c, 0)$	S_1
1	$move(a, b, 5)$	S_2
2	$move(d, T, 2)$	S_3
3	$move(a, c, 0)$	S_4
4	$move(b, c, 1)$	$S_5 = S_6$ ($d@2/0$)
5	$move(d, T, 3)$	d desliza para [3, 6) $\Rightarrow S_7$

3 Codificação CNF

3.1 Variáveis proposicionais

Para $t \in \{0, \dots, T\}$ (estado) e $t < T$ (ação). A relação *on não* é codificada; é derivada na interpretação.

Variável	Índices	Significado	Qtd. por t
$at(b, p, t)$	$p \in [0, 6 - \ell(b)]$	b começa em p	21
$lev(b, l, t)$	$l \in [0, 3]$	b está no nível l	16
$clr(b, t)$		topo de b livre	4
$cob(b, s, t)$	$s \in [0, 5]$	b cobre o slot s (aux.)	24
$ocb(b, s, l, t)$		b ocupa a célula (s, l) (aux.)	96
$ocp(s, l, t)$		alguém ocupa (s, l) (aux.)	24
$move(b, y, p, t)$	$y \in B \setminus \{b\} \cup \{T\}$	ação (somente $t < T$)	84

Total: $|V| = 185(T + 1) + 84T = 269T + 185$ (por exemplo, $T = 4$: 1261 variáveis).

3.2 Grupos de cláusulas

1. **Estado inicial e meta:** unitárias $at(b, p, 0)$, $lev(b, l, 0)$ e $at(b, p, T)$, $lev(b, l, T)$.
2. **Unicidade (A,B):** para cada b, t : $\bigvee_p at(b, p, t)$ e $\neg at(b, p_1, t) \vee \neg at(b, p_2, t)$; análogo para lev .
3. **Geometria (definições):** $cob \leftrightarrow \bigvee_{p: s \in \text{span}(b,p)} at(b, p, t)$; $ocb \leftrightarrow cob \wedge lev$; $ocp \leftrightarrow \bigvee_b ocb$.
4. **Exclusão horizontal (C):** $\neg ocb(b_1, s, l, t) \vee \neg ocb(b_2, s, l, t)$.
5. **Estabilidade (D):** para $l > 0$: $\neg at(b, p, t) \vee \neg lev(b, l, t) \vee \bigvee_{s \in S} ocp(s, l - 1, t)$ para todo subconjunto S do span com $|S| = \ell(b) - \lceil \ell(b)/2 \rceil + 1$ (“pelo menos k de n ” $\equiv$ todo subconjunto de tamanho $n - k + 1$ contém um verdadeiro).
6. **Clear (E):** $clr(b, t) \leftrightarrow \bigwedge_{s \in \text{span}} \neg ocp(s, l + 1, t)$, condicionado a $at(b, p, t) \wedge lev(b, l, t)$.
7. **Ação única (F):** $\neg move_1 \vee \neg move_2$ para todo par em t (permite ≤ 1 ação por passo; o menor T com SAT dá o plano mínimo).
8. **Pré-condições de move(b, y, p, t) (G):** $\neg m \vee clr(b, t)$; $\neg m \vee \neg at(y, p_y, t)$ para p_y sem sobreposição; para cada nível l_y de y : slots-alvo livres $\neg m \vee \neg lev(y, l_y, t) \vee \neg ocb(x, s, l_y + 1, t)$ ($x \neq b$); proibição de no-op; y no nível máximo $\Rightarrow \neg m$.
9. **Efeitos (H):** $\neg m \vee at(b, p, t + 1)$ e $\neg m \vee \neg lev(y, l_y, t) \vee lev(b, l_y + 1, t + 1)$ (ou $lev(b, 0, t + 1)$ se $y = T$).
10. **Frame (I):** $\neg at(x, p, t) \vee at(x, p, t + 1) \vee \bigvee_{m \text{ move de } x} m$; análogo para lev . Não há frame para clr (derivado).

4 Exemplos dos 3 cenários codificados para CNF

4.1 Situação 1, meta S_{f4} , $T = 4$

```
p cnf 1261 26397
13 0 <- at(c,0,0)
 4 0 <- at(a,3,0)
12 0 <- at(b,5,0)
21 0 <- at(d,3,0)
30 0 26 0 22 0 35 0 <- lev(c,0,0) lev(b,0,0) lev(a,0,0) lev(d,1,0)
```

A ação $move(d, c, 0, 0)$ é a variável 1002. Algumas de suas cláusulas:

```
-1002 41 0 # move(d,c,0,0) -> clr(d,0)
-1002 203 0 # move(d,c,0,0) -> at(d,0,1)
-1002 -16 0 # move(d,c,0,0) -> not at(c,3,0) (c nao sobrepoe o span de d)
-1002 -17 0 # move(d,c,0,0) -> not at(c,4,0)
-1002 -30 220 0 # (c no nivel 0) -> lev(d,1,1)
```

4.2 Situações 2 e 3

Mudam apenas INITIAL e GOAL (arquivo `src/cenarios.py`):

Cenário	Estado inicial	Meta principal	T^*	Cláusulas
1	$c@0/0, a@3/0, b@5/0, d@3/1$	S_{f_4}	4	26 397
2	$c@0/0, d@3/0, a@0/1, b@1/1$	S_5	5	32 752
3	mesmo do cenário 1	S_7	6	39 107

5 Mapeamento: Descrição Formal $\rightarrow$ Código

Regra formal	Código (<code>src/bw2cnf_var.py</code>)
Blocos, comprimentos, pontos	BLOCKS, LEN, MAX_POINT em <code>cenarios.py</code>
Posições válidas $0..6 - \ell(b)$	<code>positions(b)</code>
Span, sobreposição	<code>span(b,p)</code> , <code>spans_overlap</code>
<i>at</i> , <i>lev</i> , <i>clr</i> , <i>move</i>	<code>self.novo('at',b,p,t)</code> etc. (<code>criar_variaveis</code>)
Estado inicial / meta	<code>estado_inicial</code> , <code>meta_final</code>
Unicidade (A,B)	<code>axiomas_estado</code> , <code>exatamente_um</code>
Geometria	<code>geometria</code>
Exclusão horizontal (C)	<code>exclusao_horizontal</code>
Estabilidade (D)	<code>estabilidade</code>
Clear (E)	<code>clear</code>
Ação única (F)	<code>axiomas_acao</code>
Pré/efeitos (G,H)	<code>move</code> , <code>efeitos_nivel</code>
Frame (I)	<code>frame</code>
Relação <i>on</i> derivada	<code>interpretar.py:derivar_on</code>
Ordem parcial	<code>interpretar.py:ordem_parcial</code>
Oráculo	<code>oraculo_bfs.py</code>

6 Execução Passo a Passo

```
pip install python-sat # so se nao houver o binario minisat
cd src
python3 bw2cnf_var.py --cenario 1 --alvo Sf4 --horizonte 4 --saida ../cenario1
# Gerado: 1261 variaveis, 26397 clausulas
minisat ../cenario1/trab01_blocos2SAT.cnf ../cenario1/resultado1.txt
python3 interpretar.py ../cenario1/resultado1.txt --mapa ../cenario1/trab01_blocos2SAT.map --
verbose --verificar
python3 rodar_sat.py --cenario 1 --alvo Sf4 --saida ../cenario1 # busca T minimo (T=0,1,...)
python3 rodar_todos.py # todos os cenarios e alvos + resumo
```

`rodar_sat.py` usa o binário `minisat` se existir; caso contrário usa o MiniSat 2.2 do PySAT, com o mesmo formato de saída. Os resultados de todos os alvos estão em `cenarioN/<alvo>/`; os arquivos do enunciado (`trab01_blocos2SAT.cnf/.map`, `resultado1/2/3.txt`) estão em `cenario1/`, `cenario2/`, `cenario3/`.

7 Interpretação da Saída do SAT Solver

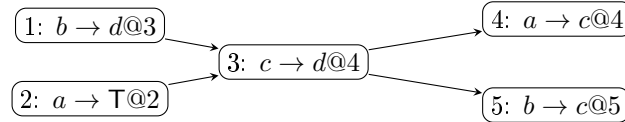
O miniSAT devolve apenas inteiros (todos os literais, com sinal). Para S_{f4} , os literais positivos que são ações são 1002, 1015, 1176, 1184, e o `.map` os traduz:

```
1002 mv(d,c,0,0) 1015 mv(a,b,5,1) 1176 mv(d,T,2,2) 1184 mv(a,c,0,3)
```

`interpretar.py` (i) lê o mapa; (ii) mantém os literais positivos; (iii) reconstrói *at, lev* por *t* e as ações; (iv) deriva *on* (nível 0 $\Rightarrow$ mesa; senão blocos do nível $l - 1$ cujo span sobrepõe o de *b*, vários $\Rightarrow$ ponte); (v) calcula a ordem parcial; (vi) com `-verificar` re-executa o plano no oráculo (independente do SAT) e testa as linearizações:

```
PLANO ENCONTRADO (4 acoes):
1. t=0: mover bloco 'd' para CIMA de 'c' em p=0
2. t=1: mover bloco 'a' para CIMA de 'b' em p=5
3. t=2: mover bloco 'd' para a MESA em p=2
4. t=3: mover bloco 'a' para CIMA de 'c' em p=0
ESTADO FINAL (t=4): a esta sobre: c ; b, c, d na MESA
ORDEM PARCIAL: 1<2 [causal: celula(slot 3, nivel 1)] 2<3 [causal: celula(slot 3, nivel 0)]
                3<4 [causal: celula(slot 0, nivel 1)]
VERIFICACAO: plano valido no oraculo = True
```

Ordem parcial no cenário S_5 . O plano do SAT é *move(b, d, 3)*; *move(a, T, 2)*; *move(c, d, 4)*; *move(a, c, 4)*; *move(b, c, 5)*. As ações 1 e 2 são independentes, e as ações 4 e 5 só dependem de 3:



Cada ordem topológica (4 no total) foi executada com sucesso no oráculo.